

Multimedia database system architecture

Hoa Dinh Nguyen, PhD.

hoand@ptit.edu.vn

Lecture Overview

- ▶ Architectural styles: centralized vs distributed MMDS
- ▶ Storage principles: separation, homogeneity, hybrid
- ▶ Server, client-server, P2P, and hybrid distributed systems
- ▶ Streaming, buffering, QoS, and network traffic characteristics
- ▶ Goal: design MMDB deployments that meet bandwidth, latency, and availability needs



Multimedia database system architecture

- ▶ Multimedia database systems often have complex architectures
- ▶ Architecture depends on application-specific requirements
- ▶ Design optimization varies according to system objectives

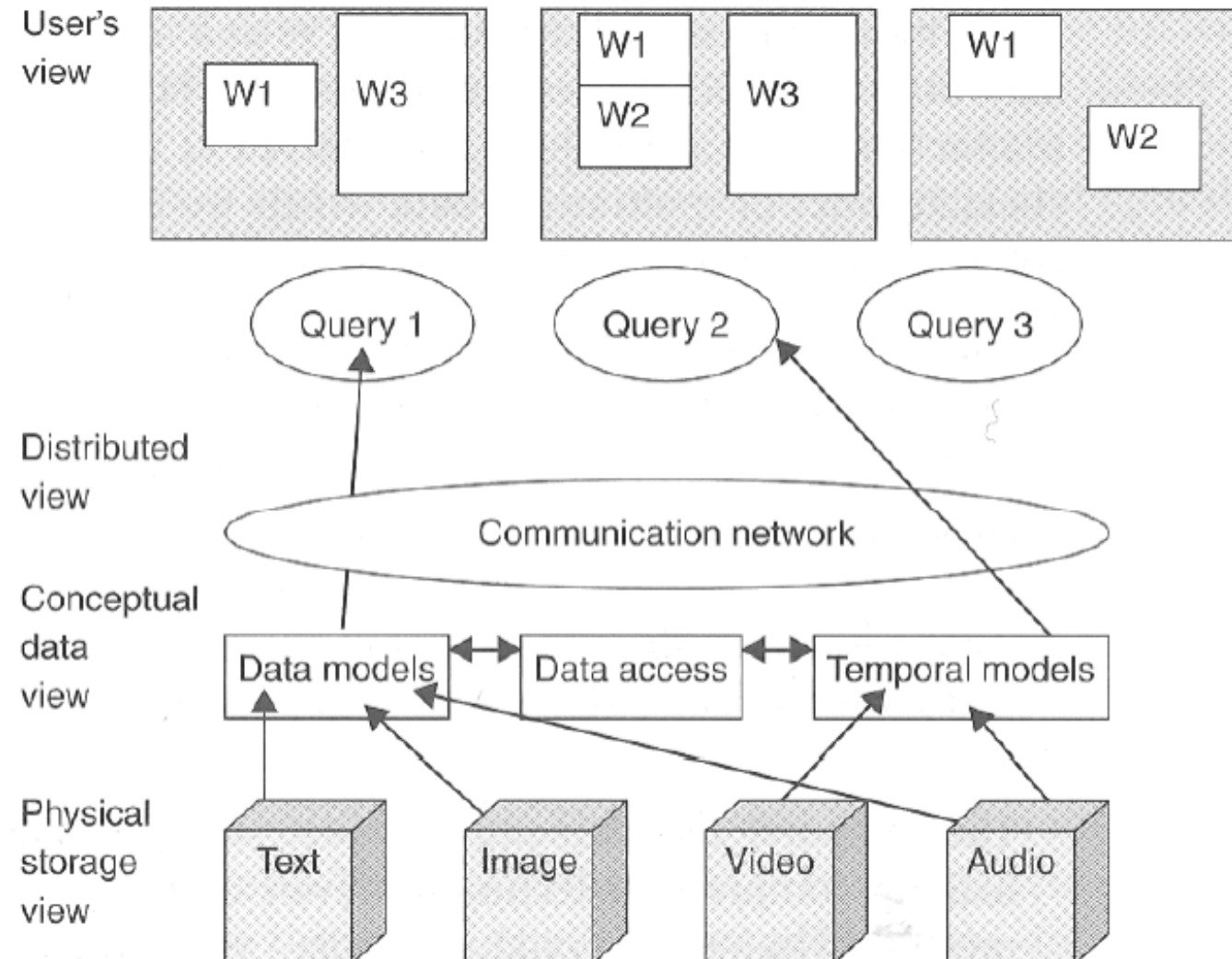
Multimedia database system architecture

- ▶ Performance depends on data transfer speed between storage and processing units
- ▶ Architecture is influenced by hardware quality and network infrastructure
- ▶ Two fundamental architectures:
 - ▶ Centralized databases
 - ▶ Distributed databases

Centralized vs Distributed MMDS Architecture

- ▶ Centralized: single DBMS site, shared storage + CPU; simple admin, single point of failure
- ▶ Suited to small/medium archives, campus media library, development prototypes
- ▶ Distributed: fragments/replicas across sites; scale storage and throughput horizontally
- ▶ Suited to Content Delivery Network-style VOD, geo-distributed users, fault tolerance requirements
- ▶ Performance bottleneck: disk I/O + network path between storage and clients
- ▶ Hybrid deployments: metadata centralized, LOBs distributed near users

Multimedia database system architecture



Choose a suitable MMDS Architecture

- ▶ For each scenario, choose centralized, distributed, or hybrid — justify:
 - University lecture capture archive (500 courses, on-campus viewers)
 - Global Netflix-style video-on-demand with edge caching
 - Hospital PACS (medical images) with strict compliance and backup
 - Collaborative video editing studio (shared timeline + raw footage)
 - Mobile news app with adaptive bitrate streaming worldwide
- ▶ Consider: user geography, fault tolerance, admin complexity, and bandwidth cost



Multimedia data storage

- ▶ **Separation principle**: different media types stored on specialized servers
- ▶ **Homogeneity principle**: all media stored using a unified data structure
- ▶ **Hybrid principle**: combination of separation and homogeneity

Storage Principles — Comparison

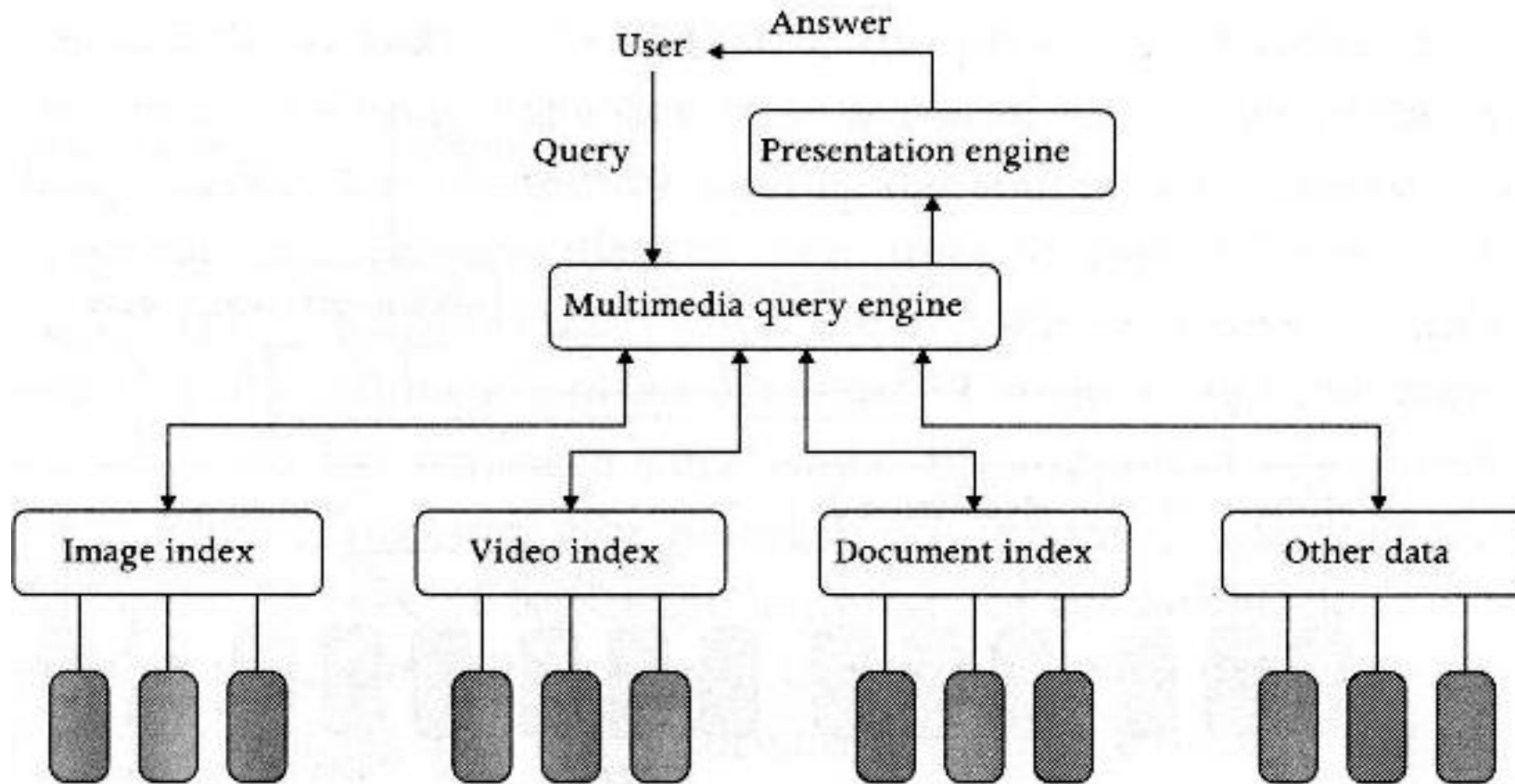
- ▶ Separation: best throughput per medium; complex integration
- ▶ Homogeneity: best developer simplicity; may miss medium-specific tuning
- ▶ Hybrid: industry default for large-scale systems
- ▶ Decision drivers: team size, service level agreement, existing DBMS, cloud vs on-prem
- ▶ Tip: justify choice with query pattern + media mix + scale



Separation Principle — Specialized Media Servers

- ▶ Each media type (text, image, audio, video) stored on dedicated server subsystem
- ▶ Video server optimized for sequential read + stream scheduling
- ▶ Image server optimized for random access + thumbnail generation
- ▶ Metadata/catalog server holds indexes, rights, and cross-media links
- ▶ Advantage: tune hardware/OS/cache per medium; isolate failure domains
- ▶ Disadvantage: cross-media transactions and unified queries harder without middleware

Multimedia database system architecture

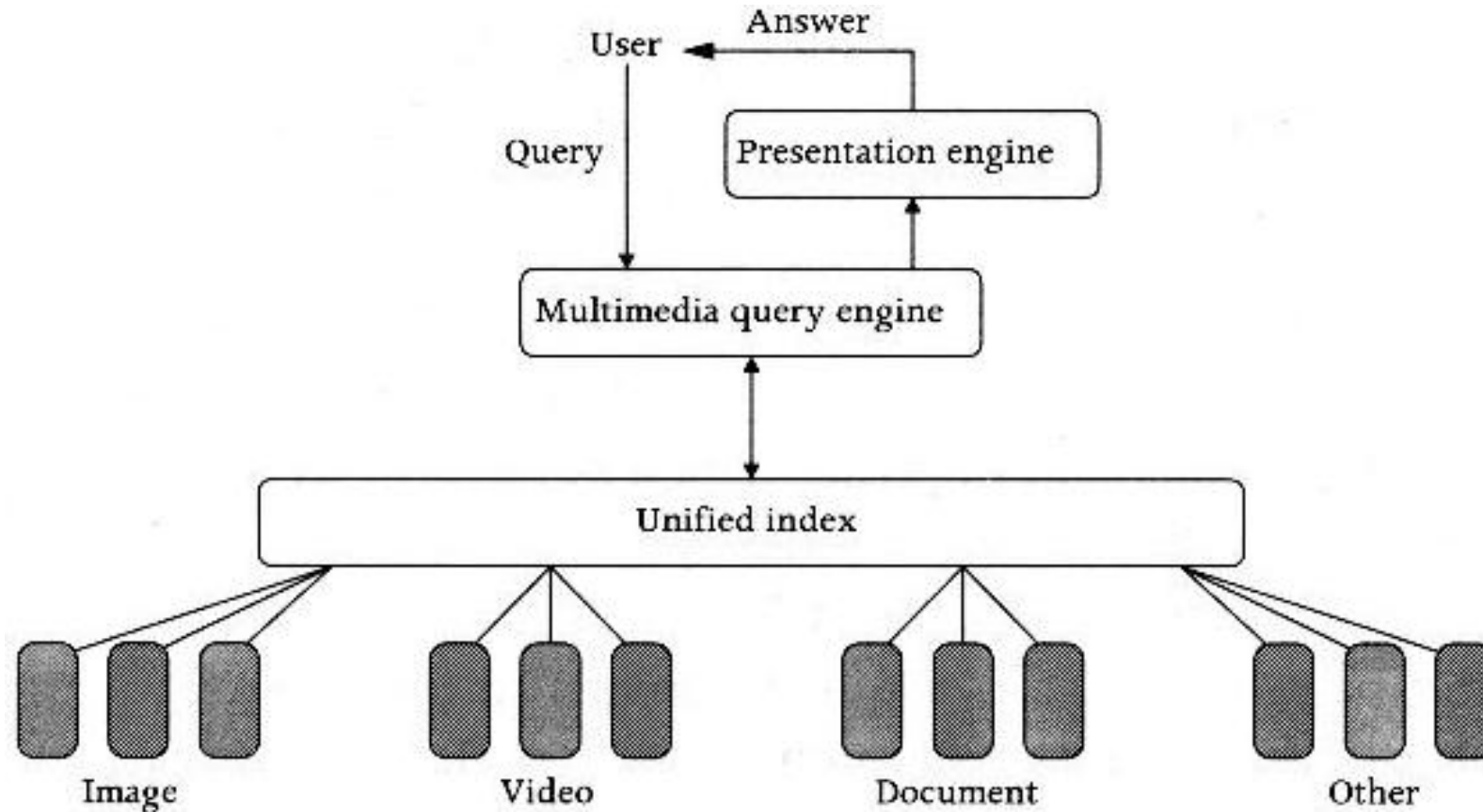


Architecture of a multimedia database under the principle of autonomy

Homogeneity Principle — Unified Storage Structure

- ▶ All media types stored under one logical schema and access Application Programming Interface.
- ▶ Single query language for metadata + blob references regardless of MIME type
- ▶ Advantage: simpler application code, ACID across media + metadata where supported
- ▶ Disadvantage: one-size storage may not optimize video streaming vs small text docs
- ▶ Often implemented as unified catalog + typed LOB columns or object store backend
- ▶ Example: one PostgreSQL + object storage bucket for all BLOBs

Multimedia database system architecture

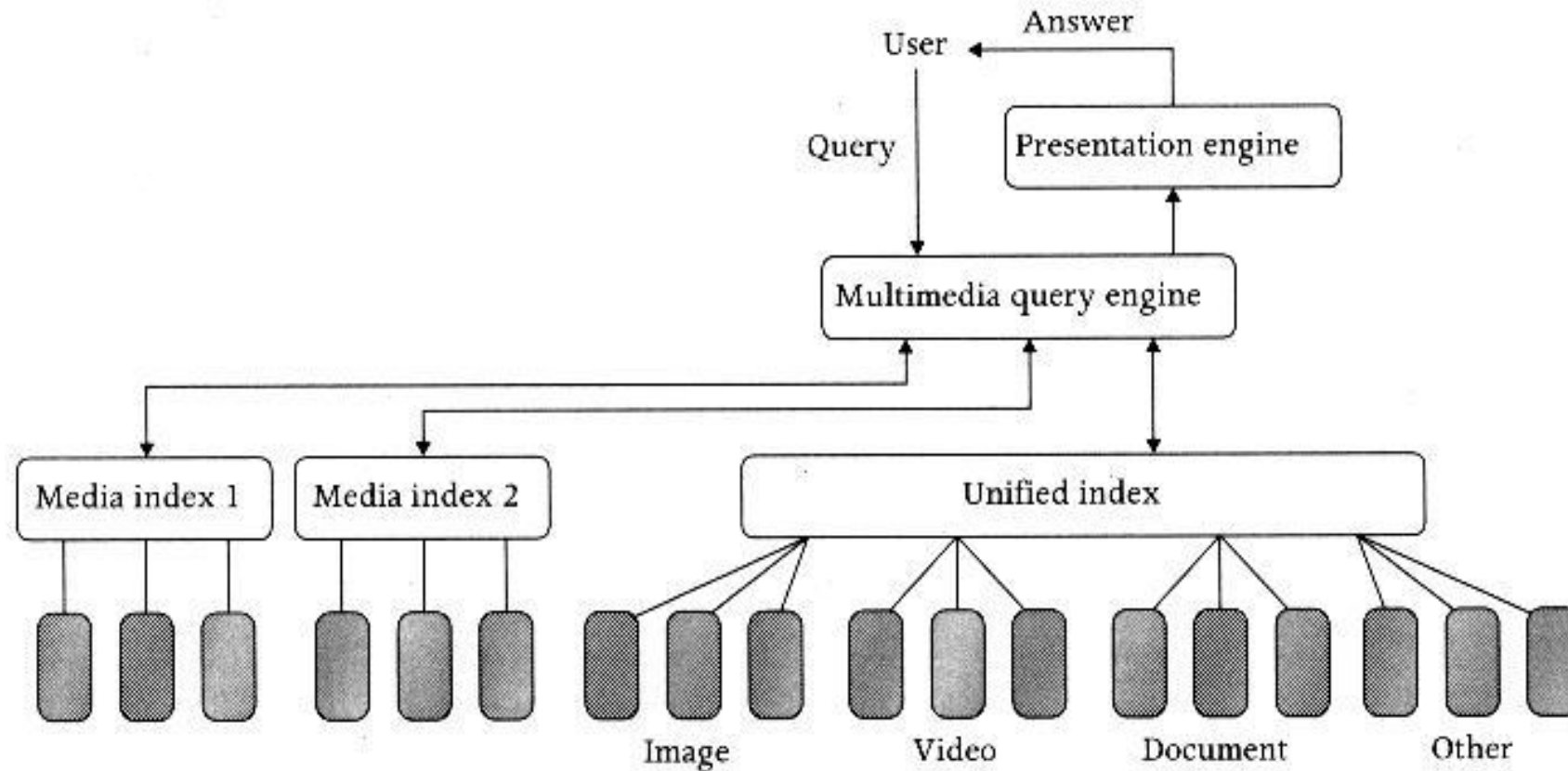


Architecture of a multimedia database under the principle of uniformity

Hybrid Principle — Combined Approach

- ▶ Unified metadata/catalog (homogeneity) + type-specialized storage engines (separation)
- ▶ Most production MMDS use this: Oracle Multimedia, MongoDB GridFS + transcode workers
- ▶ Application sees one API; physical layout may split video to stream server, images to CDN
- ▶ Middleware routes requests by MIME type and QoS class
- ▶ Supports content adaptation: transcode on read or pre-store multiple renditions
- ▶ Trade-off: more components, but balances query unity and streaming performance

Multimedia database system architecture



Architecture of a multimedia database under the principle of hybrid organization

Applications of Storage Principles

- ▶ A digital museum stores high-res images, narrated audio, and 4K exhibit videos.
 - a) Under separation: list minimum server types and one responsibility each
 - b) Under homogeneity: one schema — what tables/objects unify all three media?
 - c) Design hybrid layout: which parts centralized vs specialized?
 - d) Which principle minimizes cross-exhibit search latency?



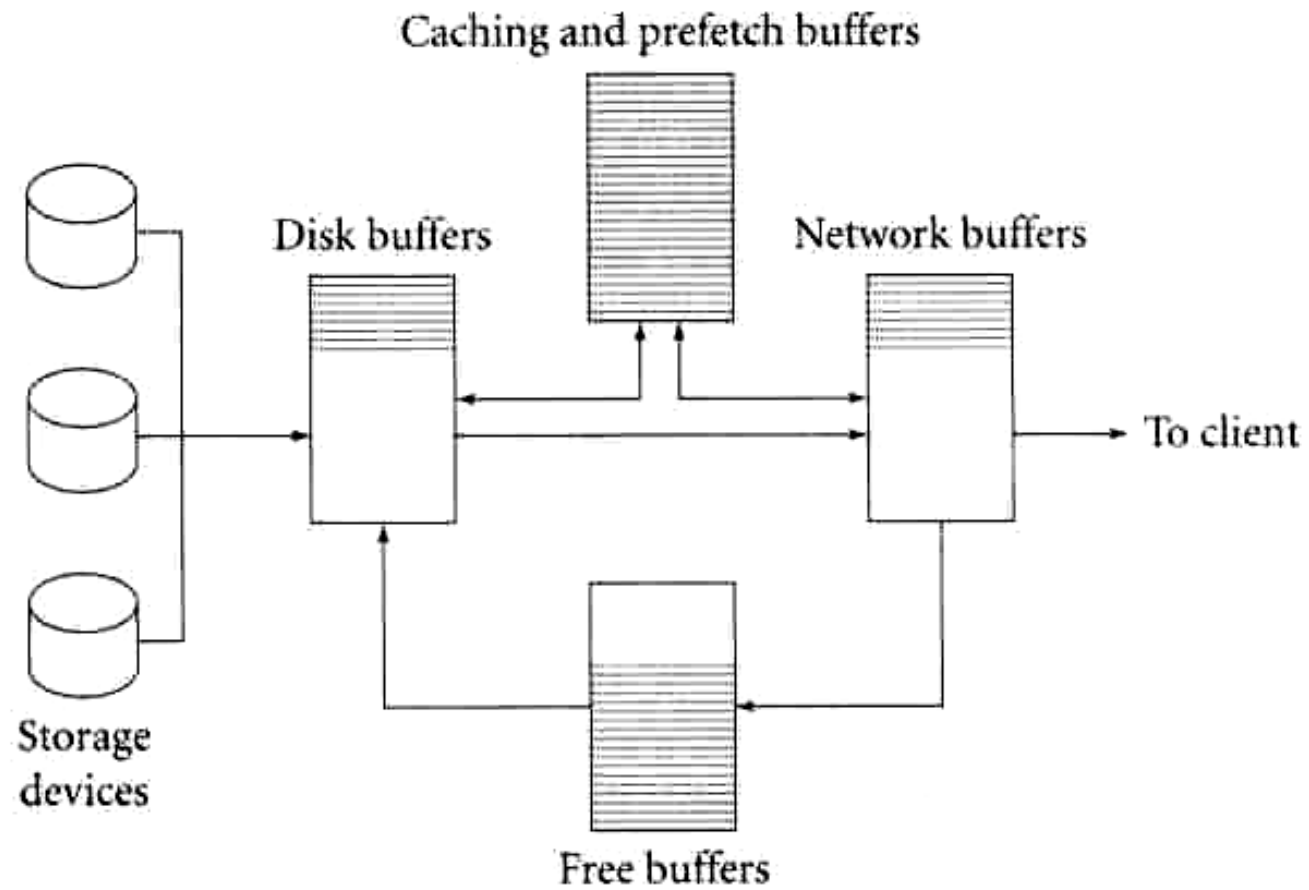
Requirements for Multimedia Servers

- ▶ Support diverse applications and users
- ▶ High access bandwidth and large storage capacity
- ▶ Support multiple multimedia data formats

Multimedia Server Architecture — Layers

- ▶ Storage layer: disks/RAID/object store; sequential layout for video
- ▶ Stream manager: schedule disk reads, merge audio/video substreams
- ▶ Catalog/index: metadata DB, search indexes
- ▶ Session manager: admission control, bandwidth reservation per client
- ▶ Cache/proxy: hot content, segment buffers at edge (CDN nodes)
- ▶ API gateway: REST/gRPC; authentication and content adaptation triggers

Multimedia servers



Multimedia Server — Key Components

- ▶ Admission controller: reject new sessions if Σ bandwidth > capacity
- ▶ Playout buffer manager: prefetch N seconds to absorb jitter
- ▶ VCR controls: pause = freeze buffer; fast-forward = skip GOP/keyframes
- ▶ Load balancer: spread sessions across replica servers
- ▶ Master server : route to best replica by load, geography, bandwidth
- ▶ High availability: replication, failover pairs, health checks



Multimedia servers

- ▶ Minimize service latency and ensure high availability
- ▶ Load balancing and real-time service support
- ▶ User interaction features: fast forward, rewind, pause

Distributed Multimedia database systems

- ▶ Databases can be horizontally or vertically fragmented
- ▶ Fragments may be replicated and stored at different locations
- ▶ Large objects (LOBs) stored near service request points

Distributed MMDS — Fragmentation and Replication

- ▶ Horizontal fragmentation: split rows by region, date, or user shard
- ▶ Vertical fragmentation: metadata local, LOBs remote
- ▶ Replication: same fragment on multiple sites for read scaling and fault tolerance
- ▶ LOB placement: store near request hotspots to cut wide-area latency
- ▶ Consistency models: strong (single primary) vs eventual (CDN replicas)
- ▶ Spatial indexes on geo-partitioned fragments



Master servers

- ▶ Retrieve data from multiple heterogeneous servers
 - ▶ Issues arise when one (or a small subset of) servers is selected by default by the system or explicitly chosen by clients
 - ▶ Static server selection strategies may lead to load imbalance
- ▶ Dynamic server selection strategies enable rotating or adaptive data retrieval from servers located at different sites.
- ▶ Multimedia data requests are routed to the most appropriate server based on the service request location, current server load, geographic distribution of servers, and available network bandwidth

Master Servers and Dynamic Selection

- ▶ Master aggregates heterogeneous backends: Oracle + file server + stream server
- ▶ Static selection: fixed mapping → hot spots and underused servers
- ▶ Dynamic selection: rotate or score servers by load, RTT, available bandwidth
- ▶ Routing inputs: client location, content popularity, replica freshness, Service Level Agreement tier
- ▶ Similar to CDN origin + edge: master = logical entry, edges = replicas



Example — Master Server Routing

- ▶ Three replicas R1 (load 80%, RTT 20 ms), R2 (40%, 50 ms), R3 (60%, 30 ms).
- ▶ Client in region closest to R2; content identical on all replicas.
 - a) Static 'always R1' policy — problem at peak?
 - b) Pick server minimizing score = $0.5 \times \text{load} + 0.5 \times \text{normalized_RTT}$ — who wins?
 - c) When route to R1 despite higher RTT?
 - d) How does replication interact with master routing?



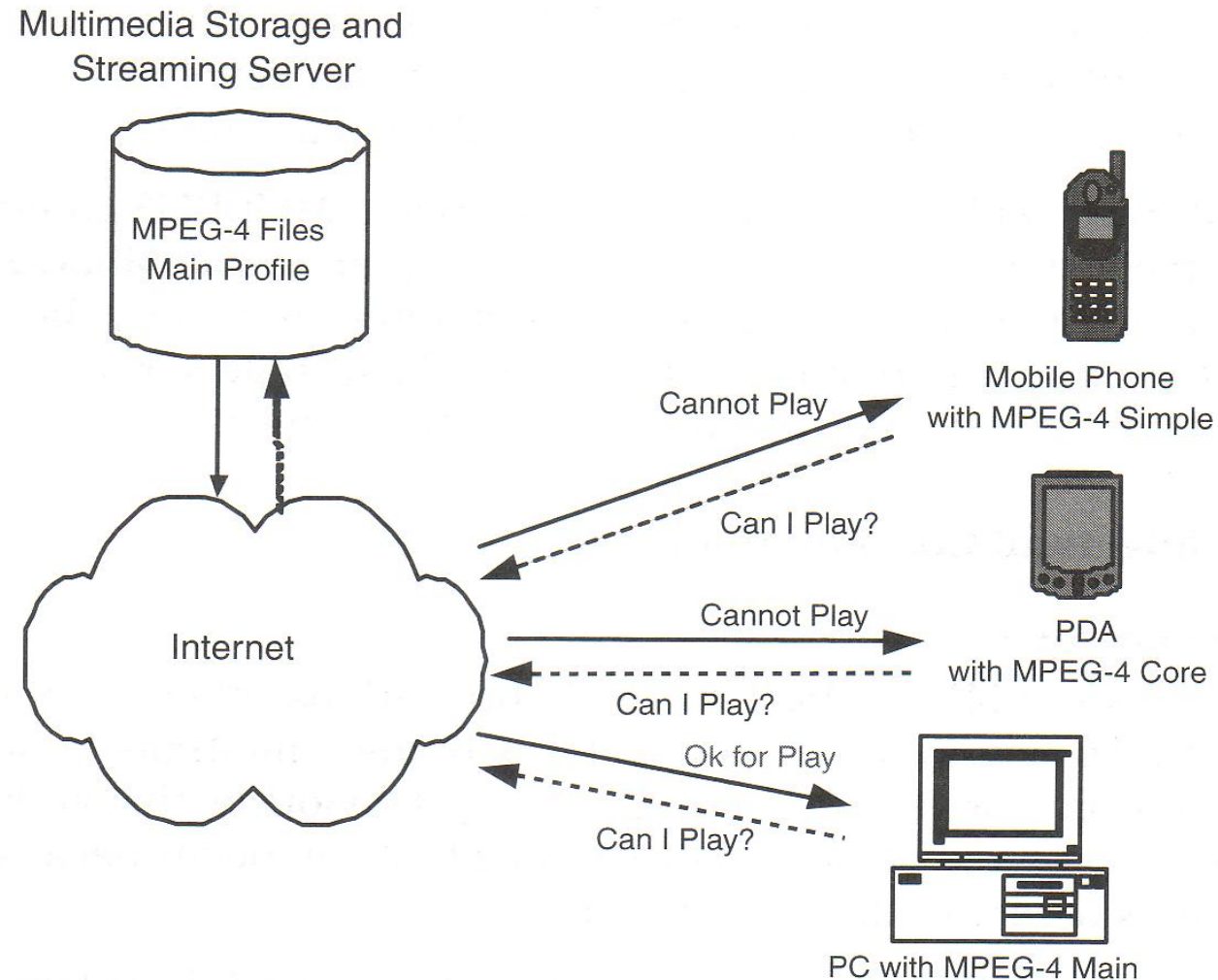
Client-server systems

- ▶ Clients send service requests to servers
- ▶ Requests processed locally at servers
- ▶ Limitations: load imbalance and heterogeneous clients
 - ▶ Large number of clients
 - ▶ Different configurations of client machines
 - ▶ Different requested data sizes

Client-Server Architecture — Tiers

- ▶ 2-tier: thick client $\leftrightarrow$ MMDB server (simple, poor adaptation for mobile)
- ▶ 3-tier: client $\leftrightarrow$ application server $\leftrightarrow$ database/stream server (common web VOD)
- ▶ n-tier: add CDN edge, transcode farm, metadata search cluster
- ▶ Separation principal limitations: many heterogeneous clients $\rightarrow$ need adaptation layer
- ▶ Server processes requests locally; bottleneck if no load balancing
- ▶ Homogeneity and hybrid principals: request down, media stream up (or separate channel)

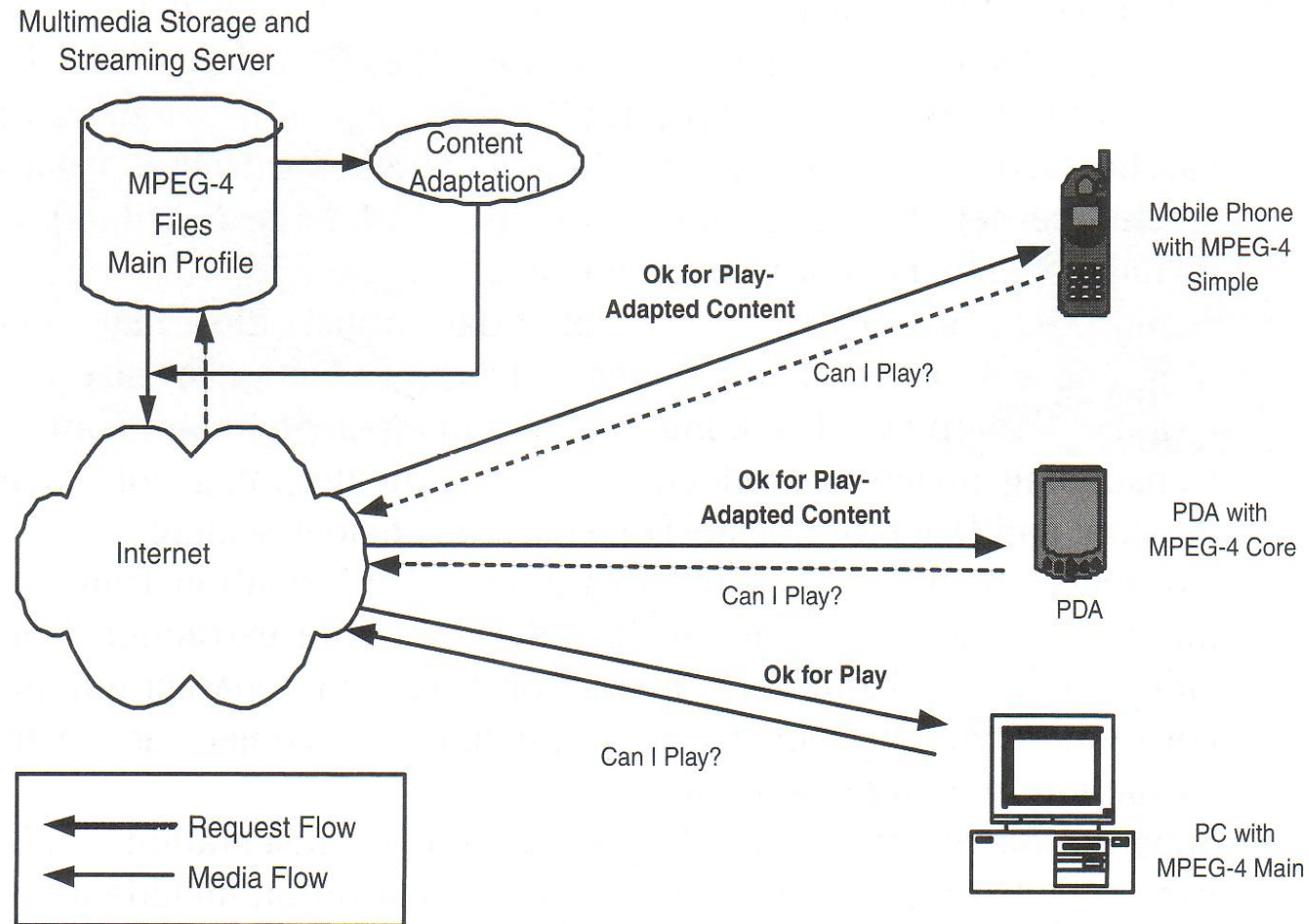
Client-server systems



Client-Server — Request and Stream Flow

1. Client: browse catalog (HTTP) → metadata query to catalog DB
2. Client: select asset → session setup to stream manager
3. Server: admission check → allocate buffer + disk bandwidth
4. Server: push RTP/HTTP segments; client playout buffer absorbs jitter
5. Control channel: pause, seek (keyframe), quality switch (ABR)
 - ▶ Separate control plane (small messages) from data plane (high bandwidth)

Client-server systems



Client-server systems

▶ Content adaptation

- ▶ Ensure service quality is appropriate for heterogeneous end-user devices
- ▶ Ensure content quality is adapted to the requirements and constraints of the service network
- ▶ Ensure information quality, including parameters such as resolution, packet rate, and related performance metrics

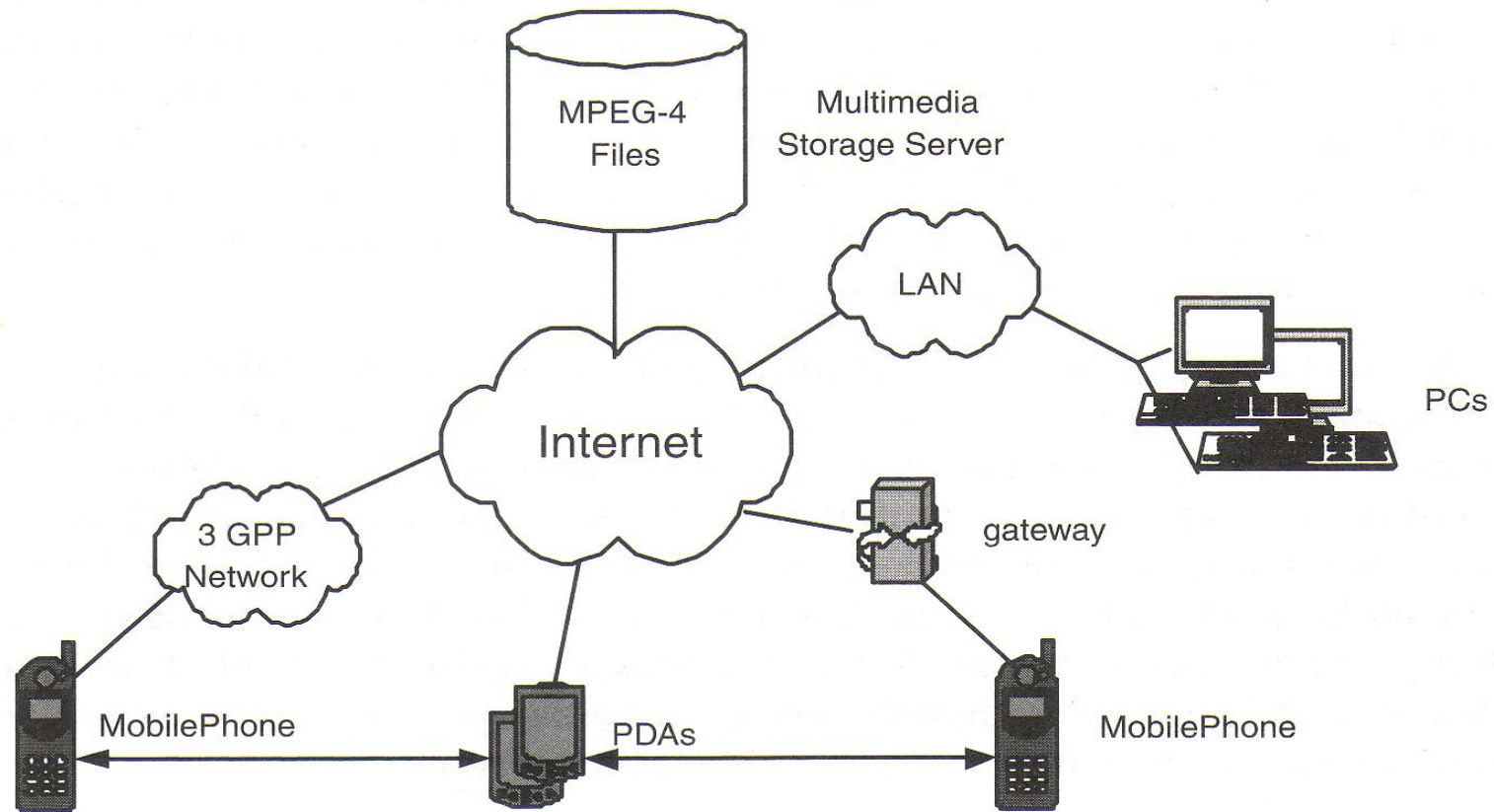
Content Adaptation and Transcoding

- ▶ Heterogeneous clients: phone vs 4K TV → different resolution, bitrate, codec
- ▶ Adaptation dimensions: resolution, frame rate, bitrate, container
- ▶ Server-side: transcode on demand or pre-generate renditions (Apple HLS ladders)
- ▶ Network-aware: reduce quality when bandwidth drops (ABR: DASH, HLS)
- ▶ Semantic preservation: metadata and chapter marks survive transcode
- ▶ Data compression: GOP structure affects seek and adaptation latency



Peer-to-peer systems

- ▶ Direct data exchange between end devices



Peer-to-peer systems

- ▶ Main features

- ▶ Buffer data before transferring
- ▶ Compatibility checking before data transfer
- ▶ Metadata generation

- ▶ Technical requirements

- ▶ P2P communication protocols, e.g., CC-PP, JXTA v1.0
- ▶ Connection protocol between mobile agents

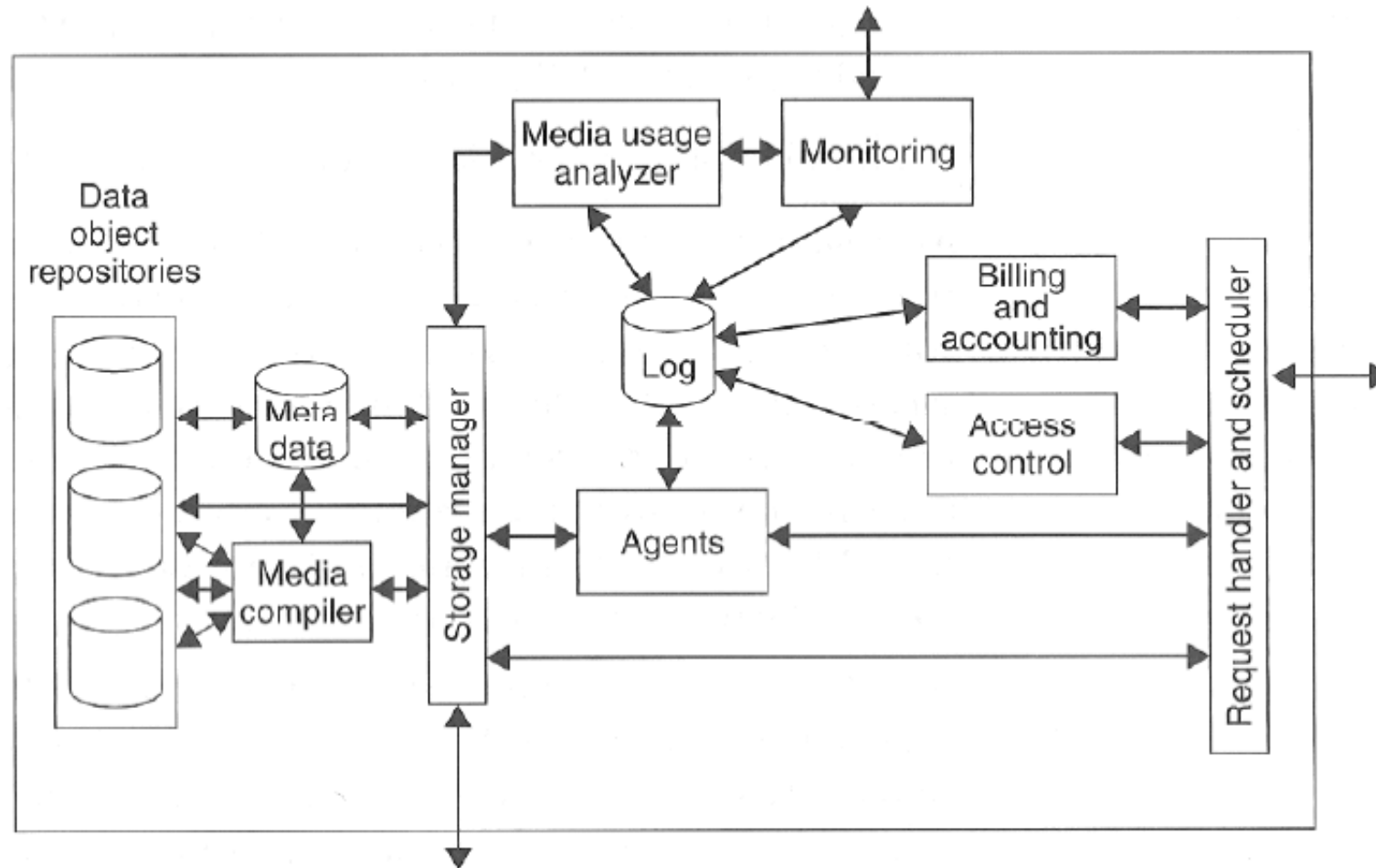
Streaming Architecture and Protocol Stack

- ▶ Application: player UI, buffer management, ABR logic
- ▶ Session: RTSP setup, HTTP range requests, WebRTC for interactive
- ▶ Transport: TCP (reliable, higher latency) vs UDP/RTP (low latency, loss)
- ▶ Network: IP, QoS marking (DiffServ), CDN anycast
- ▶ Physical: fiber, 5G, Wi-Fi — slide 3 hardware/network influence
- ▶ On-demand vs real-time — different latency/loss tolerance



Hybrid distributed systems

- ▶ There exists multiple database management systems and different data models.



Content management

- ▶ Integration of multiple management technologies
- ▶ Semantic-based management using metadata
- ▶ Automatic indexing and data partitioning mechanisms

Multimedia Data Streams

- ▶ Designed for stable and continuous data transmission
- ▶ Streaming allows content consumption during transmission
- ▶ Buffering mitigates transmission delay and jitter

Buffering, Jitter, and Playout Control

- ▶ Jitter: variance in packet arrival time — causes underrun if buffer too small
- ▶ Initial buffer fill: wait B seconds before starting playout (trade startup delay)
- ▶ Buffer compensates source–destination latency mismatch
- ▶ Underrun: stall playback; overrun: waste memory — target steady-state buffer level
- ▶ Live vs VOD: live buffer bounded by latency budget; VOD can prefetch entire file



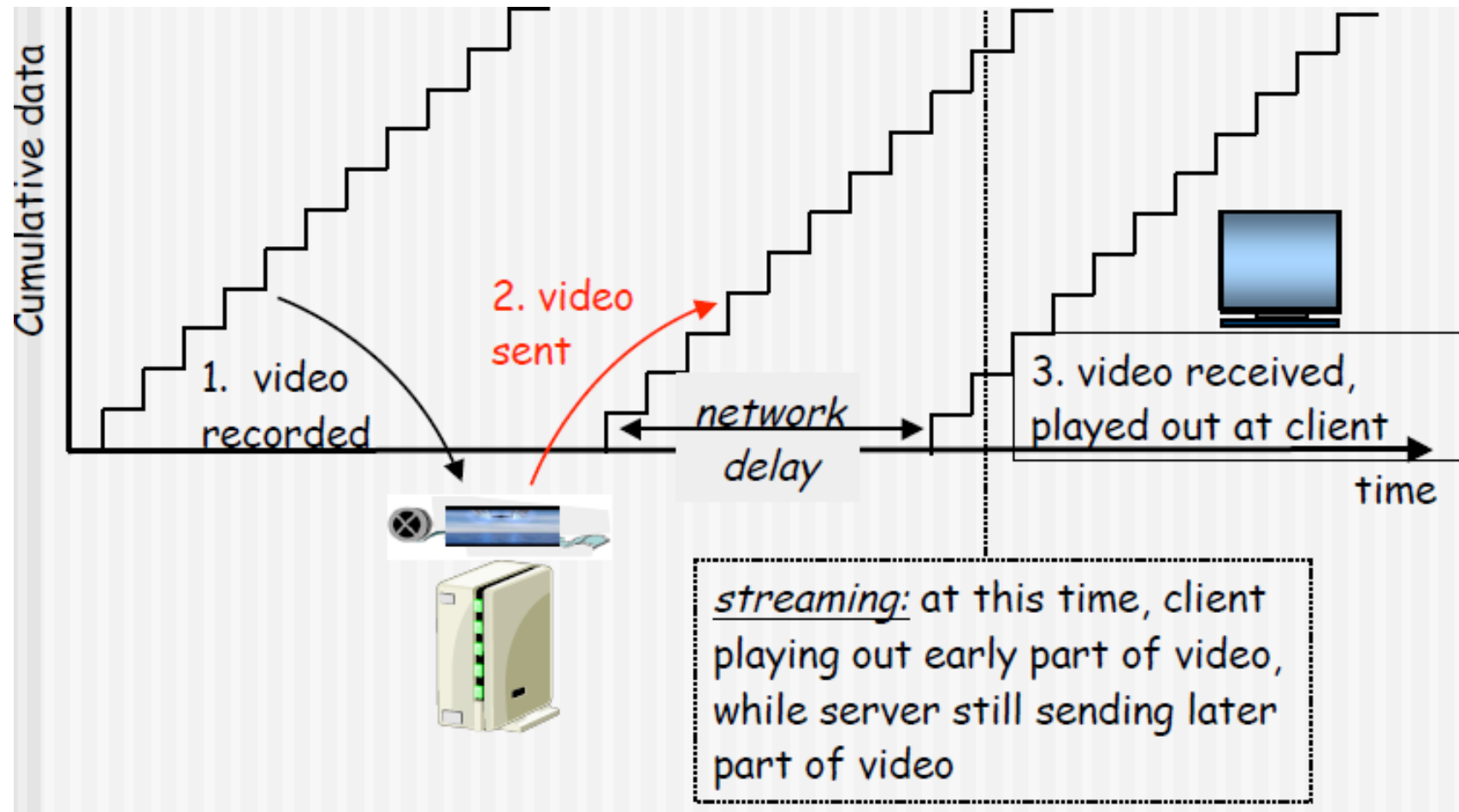
Network-based Multimedia Applications

- ▶ On-demand streaming and real-time audio/video interaction
- ▶ Sensitive to latency and packet delay
- ▶ Loss tolerance improves perceived quality of service

Data Transfer Pipeline — Server to Client

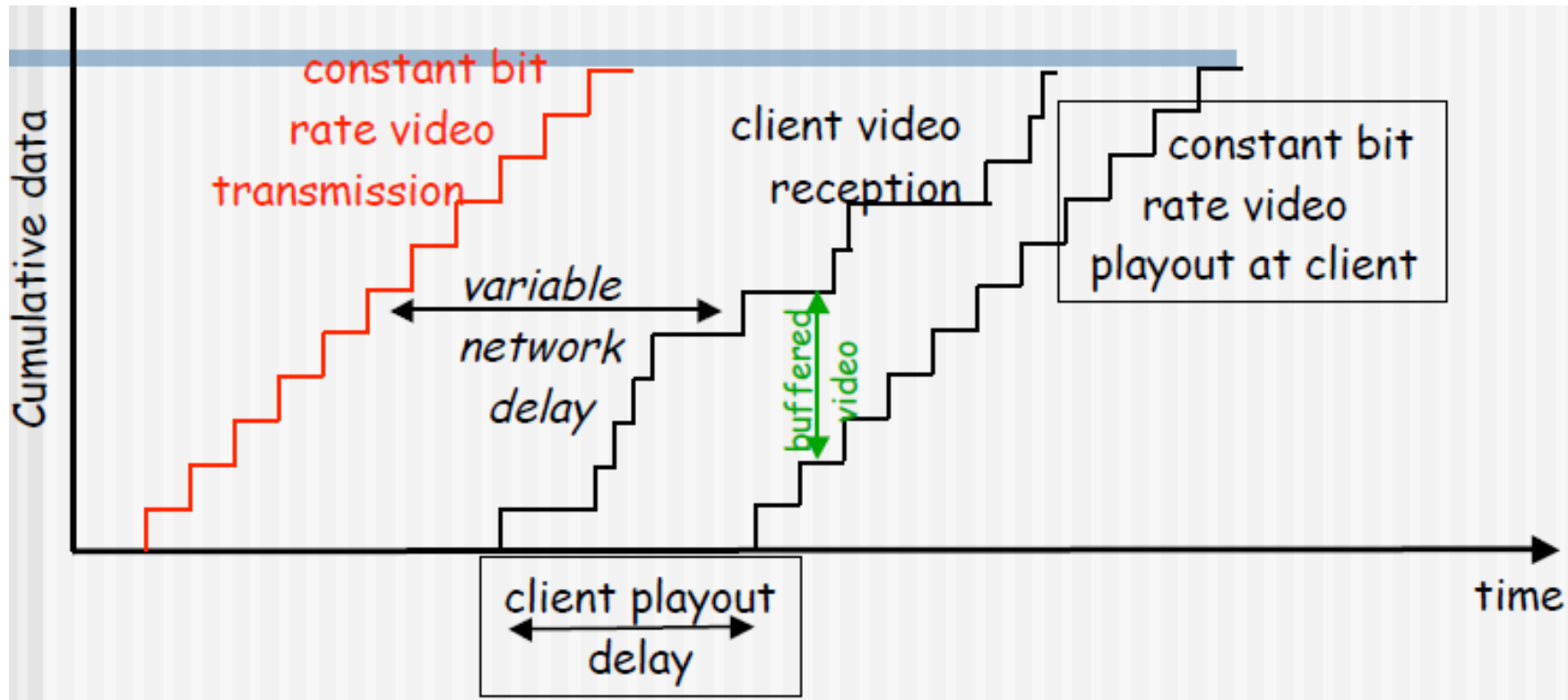
- ▶ Disk/object read → server RAM buffer → network stack → client socket buffer
- ▶ Client de-jitter buffer → decoder → display/audio output
- ▶ Parallel paths: metadata (small, TCP) vs media (large, chunked HTTP or RTP)
- ▶ Edge cache hit: skip origin disk read — serve from RAM/SSD at POP
- ▶ End-to-end delay = read + encode + queue + transmit + buffer + decode

Data transfer from servers



Data transfer using buffers

- ▶ Buffering enables the compensation of transmission latency and information delay between the source and the destination.



QoS Parameters for Multimedia Services

- ▶ Bandwidth: peak and average Mbps required per stream × concurrent users
- ▶ Latency: end-to-end delay; critical for videoconference
- ▶ Jitter: Δ between packet arrivals; drives buffer size
- ▶ Packet loss: tolerable for video (concealment); less for audio
- ▶ Availability: uptime SLA, failover
- ▶ Synchronization: lip-sync between audio and video substreams

Requirements for some multimedia services

Application	Reliability	Delay	Jitter	Bandwidth
E-mail	High	Low	Low	Low
File transfer	High	Low	Low	Medium
Web access	High	Medium	Low	Medium
Remote login	High	Medium	Medium	Low
Audio on demand	Low	Low	High	Medium
Video on demand	Low	Low	High	High
Telephony	Low	High	High	Low
Videoconferencing	Low	High	High	High

Bandwidth Requirements — Estimation

- ▶ Uncompressed SD video: ~270 Mbps; H.264 SD ~1–3 Mbps
- ▶ HD 1080p: 5–8 Mbps typical; 4K: 15–25 Mbps depending on codec
- ▶ Audio: 128–320 kbps per stream; images: burst on download
- ▶ Aggregate: $N_{\text{users}} \times \text{bitrate} \times (1 - \text{cache_hit_ratio})$ at origin
- ▶ Uplink planning: P2P reduces origin but needs peer upload capacity
- ▶ Client-server systems: map service type → minimum bandwidth table for design

Bandwidth requirements

<u>Application</u>	<u>Bandwidth</u>
Telephony	64 kbps
Telephony	16 - 32 kbps
Teleconferencing	48 - 64 kbps
2-channel audio	128 - 384 kbps
5-channel audio	320 kbps
Videoconferencing	64 - 153 kbps
High-Definition TV	17 Mbps

Characteristics of Multimedia Data Traffic

- ▶ It typically consists of continuous streams of digital video and audio data, together with digital images
- ▶ Video and audio streams are usually transmitted as separate streams
- ▶ Each stream is composed of multiple data packets
- ▶ Data packets are transmitted continuously according to a uniform transmission principle.

Summary — MMDS Architecture

- ▶ Architecture driven by scale, geography, media mix, and QoS — no single best design
- ▶ Centralized simple; distributed + CDN scales global VOD; hybrid metadata/BLOB split common
- ▶ Separation / homogeneity / hybrid govern physical vs logical storage layout
- ▶ Servers: storage, stream manager, catalog, admission control, load balancing
- ▶ Client-server dominant commercially; P2P supplements origin bandwidth
- ▶ Streaming QoS: bandwidth, latency, jitter, loss — buffer and ABR mitigate network variance

